

으CAST: 리전 용량 제약을 고려한 탄소 인지 시공간 워크로드

스케줄링

CAST: Carbon-Aware Spatio-Temporal Scheduling under Capacity Constraints

이종하 · 강희진 · 강동규 · 김현정 · 남재현*

단국대학교

초록 데이터센터의 탄소 배출량은 같은 작업이라도 어느 리전에서 언제 실행하느냐에 따라 크게 달라진다. 선행 연구는 이 잠재력을 측정했으나 대부분 리전 용량을 무제한으로 두었다. 용량을 빼면 최적해는 가장 깨끗한 리전 한 곳으로 쏠린다. 본 연구의 데이터에서도 그 조건에서는 작업의 99.75%가 프랑스 한 곳으로 몰렸는데, 이는 실제로 실행할 수 없는 배치다. 본 연구는 CAST를 제안한다. CAST는 LSTM으로 예측한 탄소집약도 위에서 정수 계획법으로 리전을 정하고, 마감 시한 안에서 실행 시각을 옮기되 리전별 용량 상한을 매 슬롯 강제한다. 탄소와 지연의 가중치는 고정하지 않고 슬롯마다 파레토 곡선의 무릎점에서 자동으로 정한다. 8개 리전, 1년치 실측 탄소 데이터, 합성 작업 146,000건으로 실험한 결과 배출량을 63.03% 줄였고 마감 위반은 없었다. 용량을 무시하면 65.93%까지 내려가지만, 그 2.9%포인트의 대가로 상한 위반이 227건에서 20,208건으로 89배 늘어난다. 가장 중요한 발견은 총계가 아니라 분포에 있다. 용량을 두지 않았을 때 시간 이동이 만드는 절감의 71.9%가 캘리포니아 한 리전에서 나오고, 그 리전에서 상한을 넘긴 시간의 87.4%가 UTC 19~21시, 곧 현지 한낮 세 시간에 몰린다. 시간 이동이 값어치를 만드는 리전과 용량이 그 값어치를 가로막는 리전이 같다.

주제어: 탄소 인지 스케줄링, 워크로드 이동, 정수 계획법, 리전 용량, LSTM

Abstract The carbon footprint of a cloud workload varies substantially with where and when it runs. Prior work has measured this potential but has mostly assumed unlimited regional capacity. Without a capacity limit the optimum collapses onto the single

cleanest region: on our data that policy sends 99.75% of all jobs to France, a placement that cannot actually be executed. We present CAST, which forecasts carbon intensity with an LSTM, places jobs across regions by integer programming, and shifts them in time within each deadline while enforcing a per-region capacity ceiling at every slot. The weight between carbon and latency is not fixed but chosen at each slot from the knee point of the Pareto curve. On 146,000 synthetic jobs driven by one year of measured carbon-intensity data across 8 regions, CAST cuts emissions by 63.03% with no deadline violations. Ignoring capacity reaches 65.93%, but those 2.9 percentage points cost an 89-fold rise in ceiling violations, from 227 to 20,208. The principal finding is not the total but its distribution: 71.9% of what temporal shifting can save without a capacity limit arises in California alone, and 87.4% of that region's over-capacity time falls in three hours, 19:00–21:00 UTC — local midday. The region where temporal shifting creates value is the same region where capacity blocks it.

Keywords: Carbon-aware scheduling, Workload shifting, Integer programming, Regional capacity, LSTM

1. 서론

데이터센터가 소비하는 전력이 빠르게 늘고 있다. 국제에너지기구는 2024년 약 415TWh였던 전 세계 데이터센터 전력 소비가 2030년 945TWh에 이를 것으로 전망했으며, AI를 가장 큰 요인으로 지목했다[1]. 에너지 사용의 투명성이 규제 대상이 된 이상[2] 소비 전력과 그에 따른 탄소 배출을 줄이는 일은 운영상의 과제가 되었고, 이미 가동

중인 인프라에 신규 설비 투자 없이 적용할 수 있는 감축 수단이 필요하다.

그런데 배출량을 결정하는 것은 소비 전력량만이 아니다. 전력 1kWh를 생산할 때 배출되는 탄소량, 곧 탄소집약도는 그 순간 전력망의 발전원 구성에 따라 정해진다. 본 연구가 다루는 8개 리전에서 2025년 실측 연평균이 인도 544.1 gCO₂/kWh, 프랑스 13.6 gCO₂/kWh로 39.9배 벌어지며, 하루 안에서도 캘리포니아는 최댓값이 최솟값의 3.53배다(365일 중앙값). 클라우드 서비스는 이미 여러 리전에 흩어져 있고 AI 학습이나 로그 처리 같은 배치 작업은 완료 시점에 여유가 있으므로, 이 격차를 쓸 조건은 갖추어져 있다.

선행 연구는 두 축 중 하나에 머물러 있다. 시간 축에서는 구글의 CICS[3]가, 공간 축에서는 CASPER[4]와 VMware GSLB[5]가 대표적인데, 이들은 한 축만 다룰 뿐 아니라 탄소와 그 밖의 목적 사이의 가중치를 고정 상수로 둔다. 두 축을 함께 다룬 Sukprasert 등의 상한 분석[6]은 공간 이동이 지배적이고 시간 이동의 추가 이득은 제한적이라고 결론지었으나, 두 축을 결합한 분석에서는 용량 제약을 두지 않았다. 용량을 명시적으로 다룬 CarbonFlex는 단일 클러스터 내 시간 이동에 한정된다[7]. 두 축을 결합하면서 용량까지 제약으로 둔 Caspian[8]은 그 대신 마감을 목적함수의 패널티로 완화한다.

용량을 빼면 무슨 일이 생기는지는 본 연구의 데이터에서 그대로 드러난다. Sukprasert 등의 정책을 원문 그대로 다시 구현해 같은 데이터에 돌리면, 작업 146,000건 가운데 145,641건 곧 99.75%가 프랑스 한 리전으로 몰린다(4.4절). 배출량은 96.61% 줄지만 용량 상한을 넘긴 배정이 120,186건에 이른다. 가장 깨끗한 리전 하나로 전부 보내는 것이 최적해가 되기 때문이며, 이는 계산 위에서만 성립하는 배치다. 용량을 제약으로 두는 순간 문제의 성격이 달라진다 — 어느 리전이 깨끗한가가 아니라, 깨끗한 리전이 받아 줄 수 있는 만큼만 보내고 남은 작업을 어디에 언제 둘 것인가가 된다.

본 논문은 탄소 인지 스케줄링 시스템 CAST를 제안한다. CAST는 세 단계로 움직인다. 예측 단계가 8개 리전의 향후 24시간 탄소집약도를 LSTM으로 산출하고, 공간 이동 단계가 정수 선형 계획법으로 실행 리전을 정하며, 시간 이동 단계가 각 작업의 마감 이내에서 실행 시각을 정한다. 배치 결정은 예측값으로 하되 배출량 산정은 실측값으로 하여 예측 오차의 대가가 결과에 반영되도록 했다.

본 논문의 기여는 두 가지다. 첫째, 공간 축의 탄소-지연 가중치를 고정하지 않고 매 슬롯 파레토 무류점에서 자동으로 정하며, 시간 축의 가중치는 작업이 신고한 지연 등급의 함수로 둔다. 둘째, 시간 이동이 리전 용량을 고려하지 않으면 동시 실행 수가 여유율 상한의 3.4배에 이른다는 것을 실측한 뒤, 이를 온라인으로 강제하는 Algorithm 1을 제시한다. 이 알고리즘은 슬롯마다 리전 점유량을 추적해 상한에 도달한 슬롯을 후보에서 빼며, 어떤 작업도 무한히 연기되지 않고 마감 위반이 생기지 않음을 증명한다. 선행 연구[7]는 이 현상을 단일 클러스터 맥락에서 지적했을 뿐 정량화하지 않았고, 리전 간 이동과 결합한 온라인 강제 기법도 제시하지 않았다.

1년치 실측 탄소집약도 데이터를 사용한 시뮬레이션에서 총 배출량이 63.03% 감소하였다. 공간 이동과 온라인 용량 인지 시간 이동을 결합한 결과로, 작업 146,000건 전량이 마감 이내에 처리되어 마감 위반은 발생하지 않았다. 용량을 무시하면 65.93%까지 내려가지만, 그 2.90%포인트의 대가로 상한 위반이 227건에서 20,208건으로 89배 늘어난다. 더 중요한 것은 그 부담이 고르게 퍼지지 않는다는 점이다. 용량을 두지 않았을 때 시간 이동이 만드는 절감의 71.9%가 캘리포니아 한 리전에서 나오고, 그 리전이 상한을 넘긴 866.9시간의 87.4%가 UTC 19~21시 세 시간, 곧 현지 한밤에 몰린다. 시간 이동이 값어치를 만드는 리전과 용량이 그 값어치를 가로막는 리전이 같다.

2. 관련 연구

선행 연구는 탄소 인지 스케줄링이 배출을 얼마나 줄일 수 있는지를 여러 방향에서 보여 왔다. 다만 그

결과를 실제로 돌아가는 시스템에 옮기려 하면 세 가지가 걸린다. 첫째, 두 축을 결합한 선행 연구[6, 8, 9]는 리전 용량과 작업 마감 가운데 하나를 양보한다. Sukprasert 등은 용량이 공간 이동의 이득을 좌우하는 변수임을 스스로 확인했음에도 두 축을 결합한 분석에서는 용량을 두지 않았고[6], Attenni 등도 자원이 항상 가용하다고 가정한다[9]. Caspian은 용량을 제약으로 두지만 그 대가로 마감을 양보하며[8], 이는 셋째에서 다르다. 탄력적 확장으로 이를 방어할 수 있으나, 그 잉여는 클라우드 전체가 집계 수준에서 갖는 성질이지 특정 시각의 특정 리전이 갖는 성질이 아니다.

Sukprasert 등(EuroSys'24) 6.4절은 공간 이득이 낮음에도 시간 변동이 커 순 절감이 유의미한 예외로 캘리포니아와 버지니아를 지목하였다[6]. 리전 용량이 유한해 공간 이동이 조기에 포화되면 부하가 고변동 리전까지 확산되므로, 이 예외가 일반적 상황이 될 수 있다.

둘째, 탄소와 그 밖의 목적 사이의 가중치가 고정 상수다. CASPER는 탄소와 서버 프로비저닝 비용 사이의 가중치를 고정하며 저자들 스스로 다른 값에서는 결과가 달라질 것이라고 밝혔다[4]. CICS[3] 역시 탄소 비용과 피크 전력 비용의 가중합을 쓰고, 두 축을 하나의 정수 계획법으로 결합한 Caspian도 가중치를 사람이 고르며 그 값을 신중하게 정해야 한다고 결론지었다[8]. 그러나 리전 간 격차는 시각에 따라 변하므로, 옮겨서 얻는 이득이 큰 시간과 그렇지 않은 시간에 같은 가중치를 쓰는 것은 최적이라 보기 어렵다.

셋째, 두 축을 실제로 결합한 Caspian조차 작업이 무한정 밀리지 않는다는 보장을 주지 않는다. Caspian은 리전과 시간대를 함께 정하고 용량도 하드 제약으로 다루지만, 마감은 목적함수의 페널티로 두는 소프트 제약이다[8]. 본 연구는 반대로 마감을 하드 제약으로 두고 용량을 양보하며, 어떤 작업도 유한 시간 안에 처리됨을 3.4절에서 증명한다. 두 시스템은 위기 상황에서 무엇을 먼저 포기할지를 정반대로 택한 것이다.

3. CAST

3.1 개요

CAST는 예측, 공간 이동, 시간 이동의 세 단계로 구성된다. 예측 단계는 리전별로 향후 24시간의 탄소집약도를 산출한다. 공간 이동 단계는 매 1시간 슬롯마다 그 슬롯에 제출된 작업의 실행 리전을 결정한다. 시간 이동 단계는 배정된 리전 안에서 각 작업의 실행 시각을 마감 이내로 결정한다. 그림 1이 세 단계와 단계 사이를 오가는 값을 보인다.

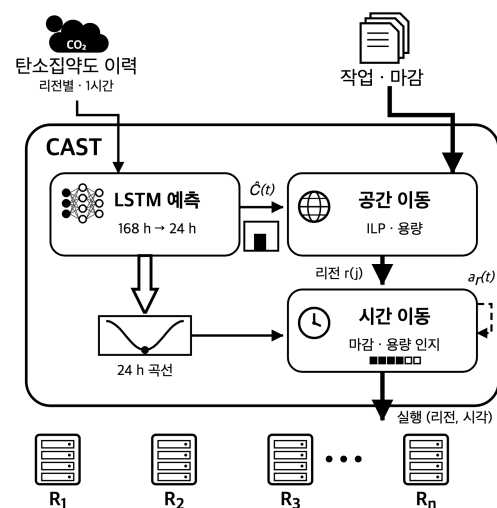


그림 1. CAST의 전체 파이프라인 — 굵은 선이 작업이 지나는 길이다. 공간 이동은 현재 값 하나로 리전을 정하고, 시간 이동은 24시간 곡선 전체를 보고, 자기가 앞서 채운 자리를 뺀 잔여 용량 안에서 실행 시각을 정한다.

설계 원칙은 셋이다. 공간 이동을 먼저 적용하고 시간 이동을 보완으로 둔다 — 선행 연구는 전 세계 70% 이상의 리전이 일간 변동성이 낮아 시간 이동 단독의 전 지구 평균 절감이 8.4%에 그친다고 보고하였다[6]. 또한 CAST에서 탄소와 지연의 가중치는 고정하지 않는다. 그리고 배치 결정은 예측값으로, 배출량 산정은 실측값으로 한다. 둘 다 예측값을 쓰면 예측 오차의 대가가 결과에서 사라지기 때문이다.

예측 모델은 리전마다 독립인 2층 LSTM이며, 은닉 크기는 64다. 과거 168시간의 탄소집약도, 무탄소·재생 전력 비율, 시각, 요일, 월의 주기 변수,

공휴일 여부를 받아 향후 24시간을 한 번에 출력한다. 캘리포니아, 텍사스, 독일의 세 리전은 풍속, 일사량, 기온을 입력에 더하고, 향후 24시간의 같은 기상 변수(24×3=72개)도 출력층에 함께 넣는다. 구조는 그림 2와 같고, 그림 3은 그 모델이 실제로 낸 예측 한 구간이다.

3.2 표기와 형식화

작업 집합을 J , 리전 집합을 R 로 둔다. 슬롯 길이는 1시간이며, 탄소집약도 데이터의 시간 해상도와 일치시킨 값이다. 작업 j 는 출발 리전 o_j , 제출 시각 s_j , 실행 시간 d_j , 지연 등급 k_j , 지연 예산 L_j 를 갖는다. 작업의 마감은 $D_j = s_j + L_j + d_j$ 다. 리전 r 은 용량 cap_r 을 가지며, 여기에 여유율 η 를 곱한 값이 동시 실행 상한이 된다. 스케줄러가 정하는 값은 두 가지다 — 실행 리전을 나타내는 배정 변수 x_{jr} (작업 j 를 리전 r 에 배정하면 1)과 실행 시각 τ_j 다.

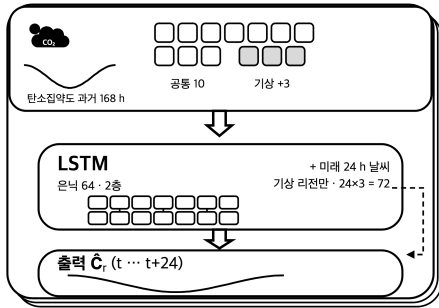


그림 2. 리전별 LSTM 구조 — 모델은 리전마다 독립이다.

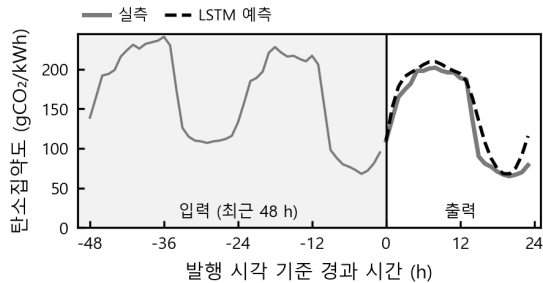


그림 3. 리전별 탄소집약도 예측 — 캘리포니아의 한 구간이다. 발행 시각 왼쪽은 입력 168시간 가운데 최근 48시간, 오른쪽은 출력 24시간이며 이 구간의 평균 절대 오차는 12.2 gCO₂/kWh다.

여기까지가 예측 모듈이다. 이제부터는 그 예측을 받아 푸는 스케줄링 문제를 적는다. 목적함수는 총

배출량이다 — 작업마다 자기가 실행된 리전에서 실행 시간 동안 내보낸 양을 구해 전부 더한 값이며, 이것을 최소화한다.

$$\min \sum_{j \in J} P \int_{\tau_j}^{\tau_j + d_j} C_{r(j)}(t) dt \quad (1)$$

여기서 $r(j)$ 는 $x_{jr} = 1$ 인 리전이고 P 는 작업의 소비 전력이다. 제약은 네 가지다.

$$\sum_{r \in R} x_{jr} = 1, \forall j \in J \quad (2)$$

$$|\{j: r(j) = r, \tau_j \leq t < \tau_j + d_j\}| \leq \lfloor \eta \cdot cap_r \rfloor, \forall r \in R, \forall t \quad (3)$$

$$s_j \leq \tau_j, \tau_j + d_j \leq D_j, \forall j \in J \quad (4)$$

$$(\tau_j - s_j) + \ell_{o_j r(j)} \leq L_j, \forall j \in J \quad (5)$$

식 (2)는 한 작업을 한 리전에만 배정한다는 뜻이다. 식 (4)는 실행이 제출 이후이고 마감 이내여야 한다는 뜻으로, 여기서 마감은 무를 수 없는 하드 제약이다 — 이를 벌점으로 무르게 둔 선행 정책에서는 마감 위반이 나온다(표 4). 식 (5)는 지연 예산을 정한다. 각 식의 항별 의미와, 식 (1)~(5)를 하나의 정수 계획으로 풀지 않은 사유는 추가자료에 실는다.

식 (3)이 본 연구의 형식화가 선행 연구와 달라지는 지점이다.

3.3 공간 이동 — 슬롯 단위 정수 계획 라우팅

공간 이동 단계는 매 슬롯마다 그 슬롯에 제출된 작업을 모으고, 리전별 탄소집약도 예측을 받아 탄소 비용과 지연 비용을 정규화한 뒤, 정수 계획을 풀어 배정을 확정하고 실행 시간 동안 해당 리전의 용량에 반영한다.

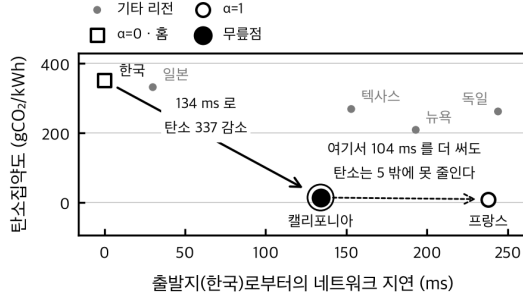


그림 4. 공간 이동이 맞바꾸는 두 축 — 한 슬롯의 여덟 리전이다(한국 출발). 무릎점은 탄소 감소가 꺾이는 자리이며, 같은 슬롯의 배정 변화는 표 1에 있다.

두 비용은 서로 다른 방식으로 정규화한다. 탄소 비용은 슬롯마다 그 슬롯의 리전별 최댓값으로 나누고, 지연 비용은 전 구간에서 고정된 값인 지연 행렬의 최댓값 244ms로 나눈다.

$$\tilde{c}_r = \frac{\hat{c}_r}{\max_{r'} \hat{c}_{r'}}, \tilde{\ell}_{or} = \frac{\ell_{or}}{244} \quad (6)$$

탄소 비용만 슬롯별로 정규화하는 것은 슬롯 간 격차의 절대 크기를 가중치 해석에서 떼어내기 위함이다. 고정된 분모를 쓰면 전 리전이 함께 깨끗한 슬롯에서 탄소 향이 일률적으로 작아져 같은 가중치가 슬롯마다 다른 뜻을 갖는다.

슬롯의 배정 문제는 다음과 같다. 배치에 속한 작업을 j , 리전을 r , 배정 변수를 x_{jr} , 미배정 변수를 z_j 로 둔다.

$$\min \sum_j \sum_r (\alpha \tilde{c}_r + (1 - \alpha) \tilde{\ell}_{or}) x_{jr} + P_{slack} \sum_j z_j \quad (7)$$

$$s.t. \quad \sum_r x_{jr} + z_j = 1 \forall j, \sum_j x_{jr} \leq a_r \forall r \quad (8)$$

a_r 은 여유율을 적용한 용량에서 현재 실행 중인 작업 수를 뺀 값이다. 미배정 변수 z_j 는 어떤 리전에도 배정할 수 없을 때 문제가 실행 불가능해지는 것을 막으며, P_{slack} 에 어떤 배정 비용보다 큰 값을 주어 배정이 가능한 한 선택되지 않도록 한다. 실측에서 미배정은 146,000건 중 한 건도 없었다.

가중치 α 는 슬롯마다 파레토 무릎점으로 결정한다. 0부터 1까지 0.1 간격의 후보 11개에 대해 각각 배정 문제를 풀어 그 배정의 평균 네트워크 지연과 예상 배출량을 얻고, 두 축을 각각 0에서 1로

정규화한 뒤 원점에 가장 가까운 점의 α 와 배정을 택한다.

$$\alpha^* = \arg \min_{\alpha} \sqrt{\tilde{\lambda}(\alpha)^2 + \tilde{\mu}(\alpha)^2} \quad (9)$$

여기서 $\tilde{\lambda}$ 와 $\tilde{\mu}$ 는 각각 0-1로 정규화한 평균 지연과 예상 배출이다.

미배정이 발생하는 후보가 있으면 미배정 수가 최소인 후보들 안에서만 고르고, 거리가 같으면 작은 α 를 택한다.

평균적으로 같은 가중치($\alpha \approx 0.508$)를 쓰는 셈이지만, 매 슬롯 다시 계산한다는 점이 차이를 만든다. 고정 $\alpha=0.5$ 는 15,718.6 kg(-46.22%, 평균 지연 29.5 ms)을, 무릎점 자동 선택은 12,609.8 kg(-56.85%, 평균 지연 36.2 ms)을 낸다. 지연을 6.7 ms 더 쓰는 대가로 탄소를 10.64%포인트 더 줄이는 셈이다.

가중치가 배정을 어떻게 바꾸는지는 한 슬롯을 열어 보면 분명하다. 그림 4의 슬롯에서 α 를 0에서 1까지 옮기면 목적지가 홈(한국), 캘리포니아, 프랑스의 세 구간으로 갈린다(표 1). 무릎점 0.508은 가운데 구간에 들어, 탄소를 홈 대비 96% 줄이면서 지연은 상한의 55%만 쓴다. 실제 배정은 리전별 잔여 용량 안에서만 이루어지므로 같은 α 라도 그 슬롯에 자리가 없으면 다음 후보로 넘어간다. 1년 전체에서 작업이 어느 리전에서 어느 리전으로 갔는지는 그림 5에 있다. 물량은 두 리전으로 모인다 — 프랑스와 캘리포니아가 전체의 62.3%를 받는다. 특히 독일은 자기 작업의 73.6%를, 인도는 71.0%를 프랑스로 보낸다.

표 1. 가중치 α 에 따른 배정 변화 — 그림 4와 같은 슬롯이다. 탄소는 gCO₂/kWh, 지연은 ms.

α	배정 리전	탄소	지연
0.0 - 0.4	한국 (홈)	351	0
0.5 - 0.9	캘리포니아	14	134
1.0	프랑스	9	238

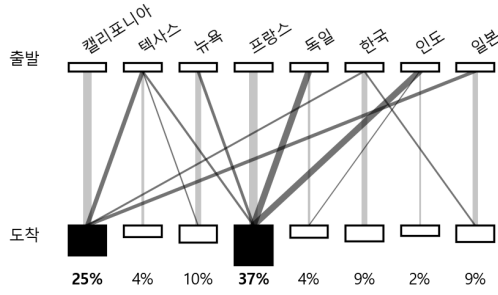


그림 5. 공간 이동이 만든 리전 간 흐름 — 1년치 146,000건의 실제 배정이다. 선 굵기가 옮긴 작업 수이며, 옮긴 물량의 91%를 차지하는 흐름 10개만 그렸다. 회색 세로선은 홈에 남은 몫이고, 20% 넘게 받은 두 리전을 검게 칠했다. 8×8 배정 행렬 전체는 추가자료에 있다.

고정 상수는 실패 방향이 슬롯에 따라 반대다. 리전 간 격차가 큰 슬롯에서는 옮길 수 있는 이득을 남긴 채 보수적으로, 격차가 작은 슬롯에서는 얻을 탄소가 적은데도 지연을 지불하며 공격적으로 동작한다. 무류점 선택은 두 축을 정규화한 곡선의 기하만 사용하므로 평가자가 정한 별도의 가중치를 요구하지 않는다. 그림 6은 고정 α 를 0부터 1까지 훑은 곡선과 슬롯별 무류점을 함께 보인다. 무류점은 슬롯마다 α 가 달라 이 곡선 위의 점이 아니며, 따라서 어떤 고정값으로도 대신할 수 없다. 또 무류점은 탄소가 가장 낮은 지점이 아니라 지연 대비 탄소 한계수익이 정점을 찍는 지점이다 — $\alpha=0.75$ 는 10,396.5 kg으로 더 낮지만 지연을 62.6 ms 더 쓴다.

3.4 시간 이동 — 용량 인지 슬롯 선택

시간 이동 단계는 작업 j 의 실행 가능 윈도우를 $[t_{early}, t_{late}] = [s_j, D_j - d_j]$ 로 둔다. 후보는 이 윈도우 안의 1시간 슬롯이다. 윈도우 안의 정수 슬롯이 제출 시각의 슬롯 하나뿐이면 옮길 자리가 없으므로 즉시 실행한다(Algorithm 1의 4번 줄). 이 경로는 용량을 확인하지 않는다 — 미룰 수 없는 작업은 막을 방법이 없기 때문이다. 후보는 예측 지평 안으로 제한한다 — 예측이 24시간까지만 있어 그 너머는 비교할 근거가 없기 때문이다.

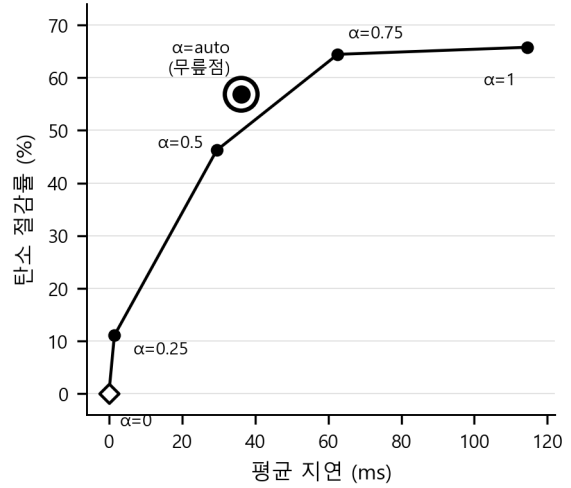


그림 6. 고정 α 스윙과 슬롯별 무류점($\alpha=auto$)의 파레토 곡선 — ◆는 baseline($\alpha=0$), ●는 무류점이다. 선은 고정 α 점만 있다.

후보 슬롯 t 의 점수는 정규화한 탄소 비용과 지연 비용의 가중합이다.

$$\tilde{C}(t) = \frac{\hat{C}(t) - C_{\min}}{C_{\max} - C_{\min}}, \tilde{D}(t) = \frac{t - t_{\text{early}}}{t_{\text{late}} - t_{\text{early}}} \quad (10)$$

$$\text{score}(t) = \beta(k)\tilde{C}(t) + (1 - \beta(k))\tilde{D}(t), \beta(k) = \frac{6-k}{5} \quad (11)$$

$\hat{C}(t)$ 는 t 부터 실행 시간 동안의 예측 탄소집약도 평균이고, $\hat{C}(t)$ 는 그것을 후보 집합 안에서 0-1로 옮긴 값이다. $C_{\min}$ 과 $C_{\max}$ 는 그 집합 안에서의 최솟값과 최댓값이다. 가중치는 작업이 신고한 지연 등급의 함수로 정의한다. 등급 k 가 작을수록, 곧 여유가 클수록 탄소 향의 비중이 커진다. 최소 등급 $k=1$ 에서 $\beta=1.0$ 이 되어 지연 향이 사라지고 가장 깨끗한 슬롯이 선택되며, 최고 등급 $k=5$ 에서 $\beta=0.2$ 가 되어 지연 향이 지배하고 사실상 즉시 실행된다.

여기까지는 각 작업이 독립적으로 자기 최적 슬롯을 고르는 규칙이다. 그러나 3.2절에서 식 (3)으로 선언한 리전별 동시 실행 상한은 아직 강제되지 않았다. 그림 7이 그 상태의 하루를 보인다 — 옮기고 싶은 저탄소 시각이 이미 상한에 닿아 있다. 이하에서 그 제약을 알고리즘 수준에서 강제한다. 식 (3)의 우변인 여유율 적용 상한 $[\eta \text{ cap}]$ 을 K 로 줄여 쓰고, 예측 지평을 H 로 둔다.

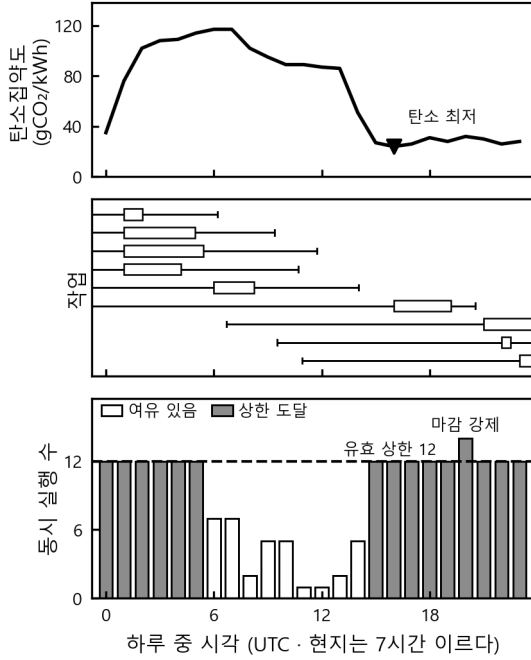


그림 7. 용량이 시간 이동을 막는 방식 — 캘리포니아의 한 날(4월)이다. 창(제출~마감)이 가장 넓은 9권을 실었으며, 가는 선이 창이고 굵은 막대가 실제 실행 구간이다. 탄소가 낮아지는 현지 한낮이 그대로 유효 상한에 닿아 옮기고 싶은 자리가 이미 차 있다.

먼저 리전 r , 시각 t 에서 남은 자리를 정의한다.

$$n_r(t) = |\{j: r(j) = r, \tau_j \leq t < \tau_j + d_j\}|, \\ a_r(t) = K_r - n_r(t) \quad (12)$$

식 (3)을 남은 자리 형태로 다시 쓴 것이다. 작업을 t 에 놓으려면 시작 시점만이 아니라 실행이 끝날 때까지 매 순간 자리가 있어야 한다.

다음으로, 슬롯 t 에서 지금이 남은 탐색 구간 안에서 최선인 작업들을 고른다.

$$Q(t) = \{j: s_j \leq t, \tau_j \text{ 미정}\}$$

$$P_r(t) = \{j \in Q(t): r(j) = r, \arg \min_{t'} \text{score}_j(t') = \max(t, s_j)\} \quad (13)$$

점수는 앞의 식 (11)을 그대로 쓰되 예측은 매 슬롯 새로 발행된 최신값을 쓰고, 지금 슬롯만은 관측값을 쓴다(추가자료 A.1.5). t 는 남은 실행 가능 윈도우 $W(t) = [\max(t, s_j), \min(D_j - d_j, t + H)]$ 안의 슬롯이다. 제출 시점의 예측을 끝까지 쓰는 사전 예약형과 달리 미래를 안다고 가정하지 않는다.

자리가 $P_r(t)$ 의 크기보다 적으면 누구를 먼저 넣을지 정해야 한다. 남은 여유가 가장 적은 작업을 먼저 넣는다 — 순수 EDF(Earliest Deadline First)다.

$$u_j(t) = (D_j - d_j) - t \\ F_r(t) = \{j \in Q(t): r(j) = r, u_j(t) < 1\} \\ R_r(t) = P_r(t) \setminus F_r(t) \text{를 } u_j(t) \text{ 오름차순 정렬} \\ S_r(t) = F_r(t) \cup R_r(t)[1 \dots \max(a_r(t) - |F_r(t)|, 0)] \quad (14)$$

$j \in S_r(t)$ 이면 $\tau_j \leftarrow \max(t, s_j)$ 로 확정한다.

세 가지를 밝혀둔다. 우선순위는 탄소 이득이 아니라 남은 여유다 — 탄소 이득으로 정렬하면 나중에 성과로 보고할 바로 그 값을 지금 순서 기준으로 쓰는 셈이라, 결과가 좋아 보이는 것이 당연해진다. $F_r(t)$ 는 대기 집합 전체에서 고른다. 그리고 $|F_r(t)|$ 가 $a_r(t)$ 보다 많으면 그 슬롯은 의도적으로 상한을 넘긴다. 본 설계에서 마감은 하드 제약이고 용량은 소프트 제약이며, 둘이 충돌하면 마감이 우선한다. 식 (3)은 일반 배정이 지켜야 할 상한이지 예외 없는 불변식이 아니다.

Algorithm 1은 이 규칙을 슬롯 단위로 반복한다.

Algorithm 1 슬롯 단위 롤링 재평가

Input: K_r , $\hat{c}_r(t)(\cdot)$, H , 작업 집합

Output: 모든 j 에 대한 τ_j

```

1:  $Q \leftarrow \emptyset$ 
2: for  $t \leftarrow 0$  to  $t_{\max}$  do
3:   for each  $j$  with  $[s_j] = t$  do
4:     if  $[D_j - d_j] \leq [s_j]$  then  $\tau_j \leftarrow s_j$ 
5:   else  $Q \leftarrow Q \cup \{j\}$ 
6:   for each  $r \in R$  do
7:      $a_r(t) \leftarrow K_r - n_r(t)$ 
8:      $F \leftarrow \{j \in Q : r(j)=r, u_j(t) < 1\}$ 
9:      $P \leftarrow \emptyset$ 
10:    for each  $j \in Q$  with  $r(j)=r, j \notin F$  do
11:       $W_j(t) \leftarrow$  실행 가능 윈도우
12:      if  $\arg \min_{t' \in W_j(t)} \text{score}_j(t') = \max(t, s_j)$  then
13:         $P \leftarrow P \cup \{j\}$ 
14:    sort  $P$  by  $u_j(t)$  ascending
15:     $S \leftarrow F \cup P[1 \dots \max(a_r(t) - |F|, 0)]$ 
16:    for each  $j \in S$  do
17:       $\tau_j \leftarrow \max(t, s_j); Q \leftarrow Q \setminus \{j\}$ 
18:    if  $Q = \emptyset$  and 남은 도착 없음 then break

```

이 알고리즘은 두 가지를 보장한다. 슬롯이 하나 지날 때마다 $u_j(t)$ 는 정확히 1씩 줄고 시작 값이 유한하므로, 유한한 슬롯 안에 $u_j(t) < 1$ 에 도달해 $F_r(t)$ 에 편입되며 용량과 무관하게 실행이 확정된다 — 어떤 작업도 무한히 연기되지 않는다. 또 후보

슬롯은 애초에 $D_j - d_j$ 이하로만 잡히므로 마감을 넘긴 배정은 탐색 공간에 존재하지 않는다.

슬롯당 계산 비용은 그 슬롯의 대기 작업 수와 예측 지평 H 의 곱에 비례한다.

비교를 위해, 같은 점수 규칙을 쓰되 용량을 전혀 보지 않는 경우도 함께 측정한다. 후보 슬롯을 고를 때 그 슬롯에 이미 배정된 작업 수를 참조하지 않으면 저탄소 시간대로 작업이 몰려도 아무것도 막지 않는다. 이 경우가 내는 값이 4장의 반사실 상한이며, 실제로 달성한 값과의 차이가 곧 용량 제약의 대가다.

3.5 탄소 회계

작업 j 의 배출량은 실행 리전의 실측 탄소집약도를 실행 구간에 대해 적분한 값에 소비 전력을 곱한 값이다.

$$E_j = P \int_{\tau_j}^{\tau_j + d_j} C_{r(j)}(t) dt \quad (15)$$

실측 계열은 1시간 해상도이므로 적분은 슬롯 단위로 근사한다 — 실행이 걸치는 슬롯들의 평균 강도에 실행 시간을 곱한다(정확한 형태는 추가자료에 둔다). 겹침 길이로 가중해 정확히 적분해도 총 배출량은 0.001% 안에서 같고 절감률은 63.03%로 바뀌지 않는다(표 2의 다른 행도 0.11%p 이내). 소비 전력은 작업당 1kW 상수로 가정한다. 실측값이 아니라 상대 비교를 위한 단순화이며, 그 결과 배출량 차이는 전적으로 실행 리전과 실행 시각에서 온다.

4. 실험과 분석

4.1 실험 설정

평가는 8개 리전의 2025년 실측 탄소집약도 1년치 위에서 수행한다. 탄소집약도는 Electricity Maps API를 통해 1시간 해상도로 수집하였다[10] — 발전과 설비를 모두 포함하는 전과정 배출계수 체계를 쓰는 공식 방법론이다[11].

워크로드는 총 146,000건이며 리전당 하루 50건이 제출된다. 도착률과 지연 등급별 분포($k=1$ 부터 3:1:1:2:3)는 실측 트레이스를 확보하지 못해 설정한

합성 파라미터다. 지연 등급 $k \in \{1, \dots, 5\}$ 는 지연 예산으로 사상된다 — $k=1$ 은 12~24시간, $k=2$ 는 2~8시간, $k=3$ 은 2~10분, $k=4$ 는 15~60초, $k=5$ 는 1~5초이며 등급 안에서는 로그균등 분포로 뽑는다.

슬롯 길이는 1시간이지만 온라인 시간 이동은 슬롯 경계가 아니라 제출 시각을 기준으로 굴러가므로, 지연 예산이 1시간보다 짧은 $k=3, 4, 5$ 도 대상에서 빠지지 않는다. 실제로 이 세 등급에서 537건이 지연되었고 마감을 넘긴 작업은 없었다.

네트워크 지연은 8×8 리전 간 왕복 지연 실측 행렬을 쓰며 최댓값은 244 ms, 비대각 최솟값은 12 ms이며, Azure 리전 간 왕복 지연 통계[12]에서 가져왔다. 리전 용량은 8개 리전에 균일하게 부여한다 — 기준 배정에서의 리전별 최대 동시 실행 수에 1.2를 곱해 16을 얻고, 여유율 0.8을 적용하여 유효 상한 12를 쓴다. 설비 용량의 실측이 아니라 기준 배정의 통계에서 유도한 값이며, 기준 배정이 실제로 도달하는 최대치(13건)의 92%다. 이 값을 바꾸면 결과가 얼마나 달라지는지는 4.3절의 용량 스위치에서 켜다. 탄소집약도의 하루 앞 예측은 선행 연구가 따로 다른 문제이며[13], 예측 모듈 자체는 본 연구의 기여가 아니므로 선행 예측 연구와의 벤치마크는 수행하지 않았다. 24시간 지평의 평균 절대 오차는 8개 리전 평균 26.1 gCO₂/kWh이며, 곡선이 평탄한 프랑스가 4.8로 가장 작고 독일 46.6과 텍사스 45.7이 가장 크다 — 예측이 어려운 리전이 곧 시간 이동의 이득이 큰 리전이다. 이 둘은 기상 입력을 실측으로 받는데도 오차가 가장 크다. 독일은 편차 자체가 가장 크고(표준편차 120.5, 다음이 텍사스 80.6), 텍사스는 시간당 변동폭이 가장 크다(16.4, 프랑스는 1.4). 이 정도 오차가 결과에 얼마나 반영되는지는 5절에서 본 연구의 완전예지 오라클, 곧 미래 탄소집약도를 모두 아는 가상의 예측기와 비교한다.

4.2 주요 결과

세 단계를 동일한 회계로 비교하였다. 홈 리전 즉시 실행이 기준이고, 여기에 탄소 인지 공간

이동을, 다시 시간 이동을 더한다. 결과는 표 2에 있고, 그림 8은 같은 값을 로그 눈금 위에서 견준다.

표 2. 단계별 총 배출량과 절감률

단계	총 배출량 (kg)	기준 대비 (%)	추가 절감 (%p)
① 홈 리전 즉시 실행	29,225.6	—	—
② 탄소 인지 공간 이동	12,609.8	-56.85	-56.85
③ ② + 시간 이동 (용량 미강제)	9,958.2	-65.93	-9.07
③' ② + 시간 이동 (사후 용량 강제)	11,873.0	-59.37	-2.52
④ ② + 시간 이동 (용량 인지, 온라인)	10,805.0	-63.03	-6.18

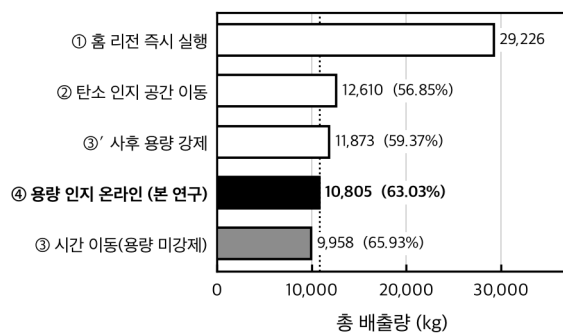


그림 8. 다섯 방식의 총 배출량 비교 (표 2) — 막대 이름은 표 2와 같다. 채운 막대가 본 연구이며, 회색 막대 ③(용량 미강제)은 달성할 수 없는 반사실이다. 세로 점선은 본 연구 ④의 값이다.

공간 이동만으로 총 배출량이 56.85% 감소한다. 시간 이동 단계가 리전 용량을 고려하지 않으면 65.93%까지 내려가지만 이는 실현 불가능한 반사실 상한이고, 배치가 끝난 뒤 사후적으로 용량을 강제하면 59.37%에 그친다. 두 경계 사이 어디에 실제 값이 있는지는 4.3절에서 확정한다. 63.03%는 작업당 1kW 가정에서 1년에 18,420.6 kg을 덜 내보낸다는 뜻이며, 실제 절감량은 작업의 소비 전력에 비례한다.

작업 146,000건은 전량 마감 이내에 처리되어 마감 위반은 발생하지 않았다. 시간 이동 단계가 후보 슬롯을 마감 이내에서만 탐색하므로 이는 설계상의 결과다. 공간 이동에 따른 평균 네트워크

지연은 36.2 ms, 95 백분위수는 137 ms이며 홈 리전 처리 비율은 56.3%다.

시간 이동의 절감은 지연 등급에 따라 극단적으로 치우친다.

절감의 93.79%가 $k=1$ 에서, 6.20%가 $k=2$ 에서 나온다. $k=3, 4, 5$ 는 지연 예산이 슬롯 길이인 1시간보다 짧아 유효 슬롯이 거의 없어 셋을 합쳐도 절감이 0.1 kg에 그치며, 이들이 전체의 60.1%인 87,712건이다. 등급별 작업 수와 절감량은 추가자료에 실는다. 슬롯 길이를 줄이면 이 셋의 기여도 늘겠으나 정수 계획을 푸는 횟수가 그만큼 늘어난다.

4.3 용량 제약과 시간 이동

시간 이동 단계가 리전 용량을 고려하지 않을 때 무슨 일이 생기는지 측정하였다. 동시 실행 수는 모든 작업의 시작·종료 시각에서의 누적합으로 세며, 로드밸런서가 용량을 산정할 때 쓰는 정의와 같다.

두 리전의 성격은 상반된다. 캘리포니아는 상한을 넘긴 866.9시간의 87.4%가 UTC 19~21시, 곧 현지 한낮 세 시간에 몰리는 반면(균등 분포라면 12.5%), 프랑스는 초과가 스물네 시간대 전부에 흩어져 있어 가장 큰 시간대도 12.1%에 그친다(캘리포니아는 일곱 시간대뿐이고 가장 큰 20시 하나가 36.0%다). 캘리포니아의 초과는 태양광이 만드는 좁은 저탄소 시간대로의 집중이고, 프랑스의 초과는 리전 전체가 항상 낮아 물량이 몰리는 것이다.

이 초과가 얼마나 손해로 이어지는지 가능하기 위해, 배치 시점에 용량을 전혀 모르는 대조군을 만들었다. 상한을 넘긴 작업을 되돌리기를 초과가 사라질 때까지 반복하는 규칙이며, 용량을 아는 스케줄러보다 항상 같거나 나쁘므로 그 결과는 아무것도 하지 않았을 때 감수할 손해의 하한이 된다 — 이하 이 값을 순진한 하한이라 부른다. 전체의 21.4%인 31,250건이 되돌려지고 총 절감률은 65.93%에서 59.37%로 내려간다. 손실은 고르지 않아 94.3%가 캘리포니아에서 나온다. 한편 시간 이동 자체가 만드는 이득도 같은 리전에 쏠린다 — 용량을 두지 않았을 때의 절감 2,651.6 kg 가운데

71.9%가 캘리포니아 몫이다. 탄소집약도가 동일 평탄한 리전에서는 실행 시각을 옮겨도 얻을 것이 없기 때문이다. 잃는 쪽과 얻는 쪽이 같은 리전이다. 리전별 손실 분해는 표 3에, 대조군의 절차는 추가자료에 실는다.

표 3. 사후 용량 강제(표 2의 ③)의 리전별 분해

리전	최대 동시	초과 시간	되돌린 작업	손실 편익	건당 손실
캘리포니아	41	866.9	16,449	1,805.1	109.74
프랑스	41	622.4	13,258	54.3	4.10
나머지 6개 리전	6~22	29.2 이하	1,543	55.4	—

주: 최대 동시는 건, 초과 시간은 시간, 손실 편익은 kg, 건당 손실은 g.

이 손해를 줄이기 위해 용량 인지 온라인 시간 이동을 구현했다. 슬롯별로 리전 점유량을 추적해 상한에 도달한 슬롯을 후보에서 미리 제외하는 방식이다. 총 배출량은 10,805.0 kg, 기준 대비 63.03% 감소로 순진한 하한(59.37%)보다 3.66%포인트 더 절감하면서, 최대 동시 실행이 41건에서 17건으로, 캘리포니아의 초과 시간이 866.9시간에서 7.25시간으로 줄어든다. 그림 9가 그 전후를 캘리포니아의 동시 실행 수로 보인다. 상한 위반은 같은 정의로 세면 용량을 무시한 반사실의 20,208건에서 227건으로 89배 줄고, 남은 227건은 마감 직전이라 강제로 편입된 106건과 옮길 슬롯이 없어 즉시 실행된 121건이다. 용량을 확인하고 배정하는 경로는 Algorithm 1의 15번 줄이 남은 자리 수만큼만 잘라내므로 정의상 위반을 낼 수 없고, 실측도 0건이다 — 구현이 알고리즘과 일치함을 보인다. 146,000건의 0.16%이고, 이 작업들은 상한을 넘겨 실행되므로 운영자는 해당 리전이 그 시각에 그만큼의 여유를 실제로 갖고 있는지만 확인하면 된다.

유효 상한을 바꾸어 보면 이 결과가 상한값에 얼마나 매여 있는지 드러난다. 상한을 6건으로 조이면 절감률이 58.05%로 내려가고, 마감이 압박해 미룰 수 없는 작업 때문에 그 상한 자체를 29,962번

넘긴다. 반대로 상한을 풀면 절감률이 65.80%까지 오르지만, 본 연구의 유효 상한 12건을 기준으로 다시 세면 위반이 25,992건이다. 본 연구가 쓰는 12건은 그 사이에서 63.03%, 227건에 해당한다.

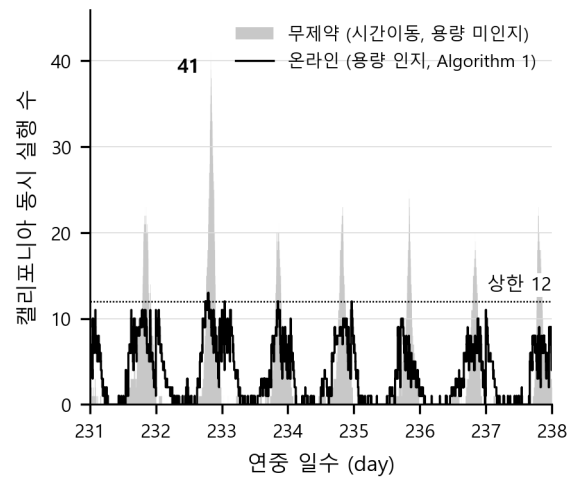


그림 9. 시간 이동 전후 캘리포니아의 동시 실행 수 — 연중 한 주(day 231–238)의 예시다. 회색 영역은 용량을 보지 않은 시간 이동, 검은 선은 온라인 용량 인지(Algorithm 1)이며, 점선은 유효 상한 12다. 온라인의 연중 최대 17건은 이 주가 아닌 다른 주에서 난다.

4.4 선행 정책 재현 비교

CICS, CASPER, Sukprasert, Caspian의 정책을 원문의 수식과 공개 코드 그대로 본 연구의 데이터 위에 다시 구현하고, 동일한 회계(식 (15))로 채점하였다. 결과는 표 4에 있고 그림 10이 같은 값을 절감률과 용량 위반의 두 축 위에 놓는다. 표 4에 실은 본 연구의 네 행은 각각 같은 종류의 선행 정책과 짝지어 읽는 자리다 — ② 공간만은 공간만 옮기는 CASPER·CICS와, ⑤ 시간만은 시간만 옮기는 Sukprasert 시간과, ③ 시간 무제약은 용량을 두지 않은 정책들과 견준다. ④ CAST (ours)가 본 연구가 실제로 내놓는 값이다. 구현 판단과 세부 파라미터는 추가자료에 정리하였다.

절감률만 보면 CASPER(95.28%)와 Sukprasert 공간+시간(96.61%)이 본 연구(63.03%)를 앞선다. 뒤엎것은 1년치 탄소집약도를 모두 미리 안다고 가정한 정책이다. 다만 본 연구를 앞서는 네 방식은 모두 상한 위반이 20,208건 이상이며, 그 안에서는

절감률이 높을수록 위반도 많다. 가장 적은 20,208건은 본 연구 자신의 반사실 상한이고, 선행 정책 가운데 가장 적은 것도 49,356건으로 본 연구의 217배다. 이 상한은 기준 배정이 실제로 도달한 최대치의 92%이므로, 이를 넘기는 배정은 그 워크로드가 한 번도 요구한 적 없는 동시 실행 수를 전제한다. CASPER의 지연·용량 제약은 본 연구의 워크로드 규모에서 한 번도 활성화되지 않아(사유는 추가자료) 매 시간 전체 요청이 그 순간 가장 깨끗한 리전 하나로 몰리며, 평균 네트워크 지연은 120.4 ms로 본 연구 무릎점의 3.3배가 된다. Sukprasert의 상한은 미래를 완전히 아는 오프라인 오라클을 전제하므로 온라인 시스템으로는 얻을 수 없는 값이며, 그 배치는 146,000건 가운데 145,641건(99.75%)을 프랑스 한 리전에 넣는다.

같은 축으로 좁혀 보면 결과가 뒤집힌다. 홈 리전을 고정하고 시간만 옮기는 조건(이하 ⑤)에서 CICS의 가상 용량 곡선은 13.76%에 그쳐 본 연구의 18.60%에 못 미친다. CICS는 하루 전에 산정한 시간별 자원 곡선 하나로 다음 날 전체를 처리해 개별 작업의 마감을 보지 못하는 반면, 본 연구는 슬롯마다 남은 여유를 다시 계산한다. 두 축을 실제로 결합한 Caspian은 30.51%를 내지만 146,000건 가운데 17,054건이 마감을 넘긴다 — 2절에서 지적한 소프트 마감의 대가가 이 숫자다. 다만 원문의 평가는 24시간 200건 규모이고 본 연구의 작업은 60.1%가 10분 이내 마감이어서, 이 수치는 원문이 겨냥한 조건 밖에서 낸 값이다.

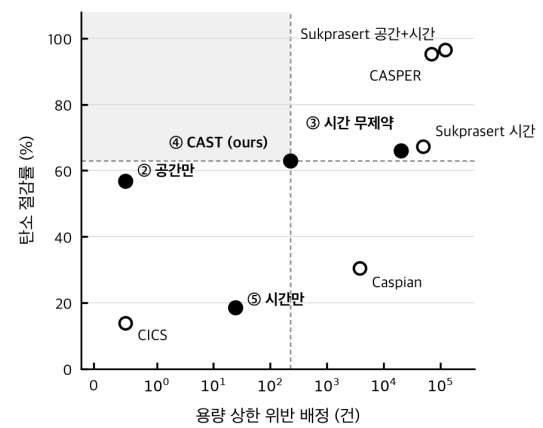


그림 10. 선행 정책 재현 결과 비교 (표 4) — 점 이름 앞의 번호는 표 2·표 4의 행 번호다. 세로축 탄소 절감률, 가로축 용량 상한 위반이다. ● 본 연구, ○ 선행 정책. 점선은 본 연구 ④의 값이며, 그보다 위반이 적으면서 더 많이 줄인 영역(음영)에는 어떤 정책도 없다.

표 4. 선행 정책을 원문 수식·공개 코드 그대로 재구현해 같은 회계로 채점한 결과 — 총배출은 kg, 위반은 건이다. 번호가 붙은 행이 본 연구이며, ①은 공통 기준이다. 용량 위반은 본 연구의 유효 상한 12건을 모든 정책에 같은 정의로 적용해 낸 값이며, 마감 위반은 Caspian을 뺀 모든 정책과 본 연구에서 0이다.

정책	총배출	절감률	용량 위반
① 기준(홈 리전 즉시)	29,225.6	—	0
CICS 가상 용량 곡선	25,203.4	13.76%	0
Caspian (분수)	20,308.4	30.51%	3,745
Sukprasert 시간	9,583.6	67.21%	49,356
CASPER CAP	1,379.1	95.28%	69,310
Sukprasert 공간+시간	990.8	96.61%	120,186
⑤ 시간만	23,789.1	18.60%	24
② 공간만	12,609.8	56.85%	0
④ CAST (ours)	10,805.0	63.03%	227
③ 시간 무제약	9,958.2	65.93%	20,208

5. 논의와 한계

이 결과가 실무에 주는 함의는 세 가지다. 첫째, 탄소 인지 스케줄링의 이득을 보고할 때 용량을 함께 보고하지 않으면 그 숫자는 비교할 수 없다. 재현한 선행 정책 가운데 본 연구보다 낮은 배출을 낸 셋은 모두 같은 상한으로 다시 세면 49,356건 이상의 초과 배정을 전제한다.

둘째, 시간 이동을 도입할지 판단하려면 리전별 탄소집약도의 평균이 아니라 변동 폭을 봐야 한다. 물량이 가장 많이 몰린 프랑스는 종일 평탄해, 용량을 두지 않았을 때 시간 이동이 내는 절감 가운데 2.3%만 났고, 이득의 대부분은 변동이 큰 캘리포니아에서 나왔다.

셋째, 예측을 더 잘하는 것보다 용량을 다루는 것이 더 큰 문제다. 예측 오차의 비용은 실제 예측(63.03%)과 본 연구의 완전예지 오라클(63.64%, 표에는 실지 않았다)의 차이인 0.61%포인트인 반면, 용량 제약의 비용은 온라인 용량 인지(63.03%)와 용량을 무시한 반사실 상한(65.93%)의 차이인 2.90%포인트로 네 배가 넘는다. 더구나 오라클 예측을 주입하면 상한 위반이 227건에서 873건으로 늘어난다 — 예측이 정확할수록 작업이 같은 저탄소 구간으로 더 일관되게 수렴하고, 그 수렴이 곧 용량 초과를 만들기 때문이다. 다만 이 배율은 워크로드에 매인다. 지연 등급 분포를 바꾸어 여섯 지점에서 재보니 0.23배에서 6.14배까지 벌어졌고, 유연한 작업의 비중이 약 27% 아래로 내려가면 순서가 뒤집힌다. 본 연구의 워크로드는 39.92%로 그 경계 위에 있다. 분포를 바꾸어도 마감 위반은 모든 지점에서, 오라클에서도 0이었다.

한계는 다섯 가지다. 첫째, 작업이 여덟 리전에 균등하게 발생하도록 두었다. 기준 배출의 69.7%가 인도(24.3%), 일본(16.3%), 한국(15.6%), 텍사스(13.6%) 네 리전에서 나오는데, 실제 클라우드 수요는 균등하지 않으므로 분포를 바꾸면 기준값과 절감 폭이 함께 달라진다. 이 측은 시험하지 못했다. 둘째, 실제 클러스터 배포가 아닌 시뮬레이션 평가여서 스케줄러와 클러스터 관리 계층의 연동 비용을 확인하지 못했다. 셋째, 캘리포니아, 텍사스, 독일 세 리전의 기상 입력은 예보 API가 아니라 해당 시각의 실측값이어서 이 세 리전의 예측 성능과 절감 효과는 실제보다 낙관적일 수 있다. 넷째, 원자료의 제약으로 예측 지평을 24시간까지만 시험했다. 다섯째, 솔버의 동물 처리 비결정성으로 재실행 시 총배출이 최대 0.14% 달라지고, 지연 등급 분포는 실측 트레이스가 아니라

설정된 값이다. 앞의 셋째 함의에서 보았듯 결과는 이 분포에 매인다.

향후 과제는 실제 예보 API 연동, 24시간을 넘는 예측 지평[14], 수요에 비례하는 용량 모델이다.

6. 결론

본 논문은 탄소 인지 스케줄링 시스템 CAST를 제안하였다. 탄소와 지연 사이의 가중치를 슬롯마다 파레토 무릎점에서 자동으로 정하고, 리전별 용량 상한을 매 슬롯 강제하면서 공간 이동과 시간 이동을 결합한다. 8개 리전의 1년치 실측 탄소집약도 위에서 배출량을 63.03% 줄였고, 작업 146,000건 전량을 마감 안에 처리하였다.

본 연구의 주된 발견은 총계가 아니라 그 분포에 있다. 용량을 두지 않았을 때 시간 이동이 만드는 절감의 71.9%가 캘리포니아 한 리전에서 발생하며, 용량 초과가 가장 심한 리전도 캘리포니아다. 그 리전에서 상한을 넘긴 시간의 87.4%는 UTC 19~21시, 곧 현지 한낮 세 시간에 몰린다. 반면 탄소집약도가 종일 평탄한 프랑스는 배정 물량이 가장 많음에도 그 절감의 2.3%만 났다. 시간 이동이 값어치를 만드는 리전과 용량이 그 값어치를 가로막는 리전이 같으며, 선행 연구가 시간 이동의 예외로 지목한 리전이 바로 그곳이었다.

참고문헌

- [1] International Energy Agency, Energy and AI, World Energy Outlook Special Report, IEA, Paris, 2025.
- [2] European Parliament and Council, Directive (EU) 2023/1791 of the European Parliament and of the Council of 13 September 2023 on Energy Efficiency (recast), Official Journal of the European Union, 2023.
- [3] A. Radovanović, R. Koningstein, I. Schneider, B. Chen, A. Duarte, B. Roy, D. Xiao, M. Haridasan, P. Hung, N. Care, S. Talukdar, E. Mullen, K. Smith, M. Cottman, and W. Cirne, “Carbon-Aware Computing for Datacenters,” IEEE Transactions on Power Systems, Vol. 38, No. 2, pp. 1270-1280, Mar. 2023.
- [4] A. Souza, S. Jasoria, B. Chakrabarty, A. Bridgwater, A. Lundberg, F. Skogh, D. Irwin, P. Shenoy, and A. Ali-Eldin, “CASPER: Carbon-Aware Scheduling and Provisioning for Distributed Web Services,” Proc. of the 14th Int. Green and Sustainable Computing Conf. (IGSC), Toronto, Canada, 2023.
- [5] D. Maji, B. Pfaff, V. P. R. Sreenivasan, V. Firoiu, S. Iyer, C. Josephson, Z. Pan, and R. K. Sitaraman, “Bringing Carbon Awareness to Multi-Cloud Application Delivery,” Proc. of the 2nd Workshop on Sustainable Computer Systems (HotCarbon), Boston, USA, pp. 1-6, Jul. 2023.
- [6] T. Sukprasert, A. Souza, N. Bashir, D. Irwin, and P. Shenoy, “On the Limitations of Carbon-Aware Temporal and Spatial Workload Shifting in the Cloud,” Proc. of the 19th European Conf. on Computer Systems (EuroSys), Athens, Greece, pp. 924-941, Apr. 2024.
- [7] W. A. Hanafy, L. Wu, D. Irwin, and P. Shenoy, “CarbonFlex: Enabling Carbon-aware Provisioning and Scheduling for Cloud Clusters,” arXiv:2505.18357, 2025.

- [8] T. Bahreini, A. N. Tantawi, and O. Tardieu, "Caspian: A Carbon-aware Workload Scheduler in Multi-Cluster Kubernetes Environments," Proc. of the 32nd IEEE Int. Symp. on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS), Krakow, Poland, pp. 1-8, Oct. 2024.
- [9] G. Attenni, Y. Moawad, N. Bartolini, and L. Thamsen, "Spatio-Temporal Shifting to Reduce Carbon, Water, and Land-Use Footprints of Cloud Workloads," arXiv:2512.08725, 2025.
- [10] Electricity Maps. (2026, Sep. 22). Data Portal [Online]. Available: <https://www.electricitymaps.com/data-portal>
- [11] Electricity Maps. (2026, Sep. 22). Methodology [Online]. Available: <https://www.electricitymaps.com/data/methodology>
- [12] Microsoft. (2026, Sep. 22). Azure Network Round-Trip Latency Statistics [Online]. Available: <https://learn.microsoft.com/en-us/azure/networking/azure-network-latency>
- [13] D. Maji, R. K. Sitaraman, and P. Shenoy, "DACF: Day-Ahead Carbon Intensity Forecasting of Power Grids using Machine Learning," Proc. of the 13th ACM Int. Conf. on Future Energy Systems (e-Energy), pp. 188-192, Jun.-Jul. 2022.
- [14] D. Maji, P. Shenoy, and R. K. Sitaraman, "CarbonCast: Multi-Day Forecasting of Grid Carbon Intensity," Proc. of the 9th ACM Int. Conf. on Systems for Energy-Efficient Buildings, Cities, and Transportation (BuildSys), Boston, USA, pp. 198-207, Nov. 2022.